

Exp No: 3 IMPLEMENTING ASSET MANAGEMENT WITH HYPERLEDGER FABRIC

Aim

To implement an Asset Management application using Hyperledger Fabric by creating, reading, updating, transferring, and deleting assets through JavaScript chaincode and verifying the transactions on the blockchain ledger.

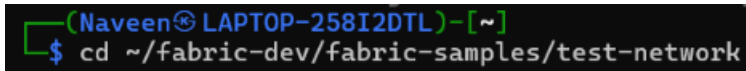
Requirements

- Kali Linux / Ubuntu
- Docker and Docker Compose
- Node.js (Version 18 or later) and npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images

Procedure

1. Navigate to the Fabric Test Network

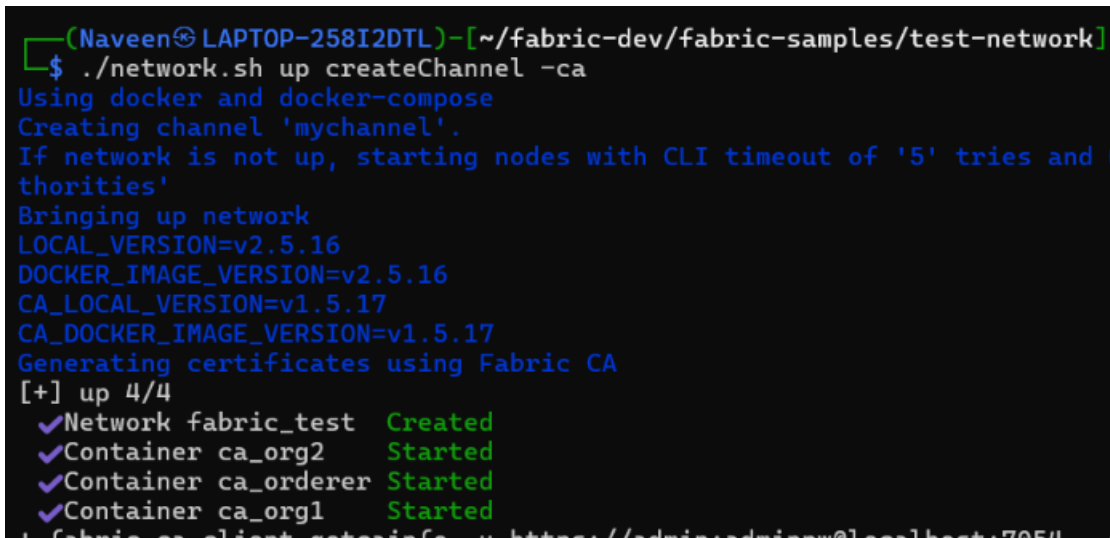
```
cd ~/fabric-dev/fabric-samples/test-network
```



```
(Naveen@LAPTOP-258I2DTL)-[~]  
$ cd ~/fabric-dev/fabric-samples/test-network
```

2. Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```



```
(Naveen@LAPTOP-258I2DTL)-[~/fabric-dev/fabric-samples/test-network]  
$ ./network.sh up createChannel -ca  
Using docker and docker-compose  
Creating channel 'mychannel'.  
If network is not up, starting nodes with CLI timeout of '5' tries and  
authorities'  
Bringing up network  
LOCAL_VERSION=v2.5.16  
DOCKER_IMAGE_VERSION=v2.5.16  
CA_LOCAL_VERSION=v1.5.17  
CA_DOCKER_IMAGE_VERSION=v1.5.17  
Generating certificates using Fabric CA  
[+] up 4/4  
✓Network fabric_test Created  
✓Container ca_org2 Started  
✓Container ca_orderer Started  
✓Container ca_org1 Started  
+ fabric-ca-client getinfo -u http://admin:adminpw@localhost:7054/
```

3. Deploy the Asset Transfer Chaincode

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-  
basic/chaincode-javascript
```

4. Initialize the Ledger

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"InitLedger","Args":[]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT" \
--waitForEvent
2026-09-01 16:38:38.432 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [0c1
3d673e0310fbe8c3b5c54b323839985d1b5da1bfa7a29604ed375a1e82c51] committed wit
h status (VALID) at localhost:7051
2026-09-01 16:38:38.433 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [0c1
3d673e0310fbe8c3b5c54b323839985d1b5da1bfa7a29604ed375a1e82c51] committed wit
h status (VALID) at localhost:9051
2026-09-01 16:38:38.433 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery
-> Chaincode invoke successful. result: status:200
```

5. Query All Assets

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":
5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owne
r":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green",
"ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValu
e":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"ass
et"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana",
"Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asse
t6","Owner":"Michel","Size":15,"docType":"asset"}]
```

6. Create a New Asset

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT" \
--waitForEvent
2026-09-01 16:41:09.232 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [052d5d5c95f0e060f34f04930c56eff9d0f6d49c78a32e20505cfbe5e2cb1e97] committed with status (VALID) at localhost:9051
2026-09-01 16:41:09.233 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [052d5d5c95f0e060f34f04930c56eff9d0f6d49c78a32e20505cfbe5e2cb1e97] committed with status (VALID) at localhost:7051
2026-09-01 16:41:09.233 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\\"ID\\":\\"asset11\\",\\"Color\\":\\"White\\",\\"Size\\":12,\\"Owner\\":\\"Arun\\",\\"AppraisedValue\\":900}"
```

7. Read the Asset Details

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
{"AppraisedValue":900,"Color":"White","ID":"asset11","Owner":"Arun","Size":12}
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

8. Update Asset Information

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c '{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT" \
--waitForEvent
2026-09-01 16:44:24.847 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [1aa7d45ffffae24877f961a33cdd995e38069a4124d32bc743c457d2174fa9538] committed with status (VALID) at localhost:7051
2026-09-01 16:44:24.847 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [1aa7d45ffffae24877f961a33cdd995e38069a4124d32bc743c457d2174fa9538] committed with status (VALID) at localhost:9051
2026-09-01 16:44:24.848 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\\"type\\":\\"Buffer\\",\\"data\\":[]}"
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

9. Transfer Asset Ownership

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"TransferAsset","Args":["asset11","David"]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"TransferAsset","Args":["asset11","David"]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT" \
--waitForEvent
2026-09-01 16:46:07.533 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [2af
b0216ad1d9319084cd9805a36c5d5e3afd9f6a079698c7f976c7995dd3b11] committed wit
h status (VALID) at localhost:9051
2026-09-01 16:46:07.533 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [2af
b0216ad1d9319084cd9805a36c5d5e3afd9f6a079698c7f976c7995dd3b11] committed wit
h status (VALID) at localhost:7051
2026-09-01 16:46:07.533 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery
-> Chaincode invoke successful. result: status:200 payload:"Arun"
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

10. Verify the Updated Asset

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
{"AppraisedValue":"1200","Color":"Black","ID":"asset11","Owner":"David","Siz
e":"15"}
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

11. Delete the Asset

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
```



```
'{"function":"DeleteAsset","Args":["asset11"]}'
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"DeleteAsset","Args":["asset11"]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT" \
--waitForEvent
2026-09-01 16:47:34.561 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [8a27b4e4be9d6a1f7bdacce776648d9db67edff2795af69f2979e78669b0bd76] committed with status (VALID) at localhost:7051
2026-09-01 16:47:34.563 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [8a27b4e4be9d6a1f7bdacce776648d9db67edff2795af69f2979e78669b0bd76] committed with status (VALID) at localhost:9051
2026-09-01 16:47:34.563 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\\\"type\\\":\\\"Buffer\\\",\\\"data\\\":[]}"
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

12. Verify Asset Deletion

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset11 does not exist"
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

13. Stop the Network

```
./network.sh down
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 10/10
Container ca_org2 Removed 3.65s
Container peer0.org2.example.com Removed 0.95s
Container peer0.org1.example.com Removed 1.05s
Container ca_orderer Removed 3.75s
Container orderer.example.com Removed 0.55s
Container ca_org1 Removed 3.45s
Network fabric_test Removed 0.25s
Volume compose_peer0.org1.example.com Removed 0.05s
Volume compose_peer0.org2.example.com Removed 0.05s
Volume compose_orderer.example.com Removed 0.05s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908e0fd355f864cfa1398b52d341cb:latest
Deleted: sha256:95323b6d7715a639f169827f0a03f5a71aba5fecaf48f6dfff72787c1ed121a04
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:4d07f56553b266dc1bbbc40d2c474652d648cd98d4e2ce3604f802323ccb1ac0
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

After creating asset11:

```
Asset asset11 created successfully
```

```
{"ID":"asset11","Color":"White","Size":12,"Owner":"Arun","AppraisedValue":900}
```

After updating and transferring ownership:

```
{"ID":"asset11","Color":"Black","Size":15,"Owner":"David","AppraisedValue":1200}
```

After deletion verification:

```
Error: asset11 does not exist
```

Result

The Asset Management application was successfully implemented using Hyperledger Fabric. The asset lifecycle operations, including asset creation, querying, updating, ownership transfer, and deletion, were executed successfully. The transactions were recorded on the blockchain ledger, demonstrating secure and immutable asset management in a permissioned blockchain network.